



SEGURIDAD EN SISTEMAS LINUX

CAPÍTULO

Fundamentos

Privilegios, Capacidades
y Control de Acceso

Administración Avanzada de Linux • Módulo de Seguridad

Contents

1	Fundamentos: Privilegios, Capacidades y Control de Acceso	2
1.1	Privilegios	2
1.1.1	Modo usuario vs. modo kernel	2
1.1.2	Identidades y credenciales en el kernel: UID/GID y <code>struct cred</code>	4
1.1.3	Capacidades POSIX (<i>POSIX Capabilities</i>)	6
1.1.4	Permisos, ACL y atributos extendidos	9
1.1.5	Protecciones relacionadas: <code>nosuid</code> , <code>noexec</code> , <code>nodev</code>	11
	Resumen del capítulo	13

Chapter 1

Fundamentos: Privilegios, Capacidades y Control de Acceso

Este capítulo sienta las bases conceptuales y prácticas del modelo de seguridad de Linux. Se estudia cómo el kernel distingue niveles de ejecución, cómo representa las identidades de procesos, el modelo de capacidades POSIX que reemplaza el binomio *root/no-root*, el sistema clásico de permisos junto con las ACL y atributos extendidos, y finalmente las opciones de montaje que refuerzan la seguridad a nivel de sistema de archivos.

1.1 Privilegios

Los **privilegios** son los derechos que posee un proceso para realizar operaciones protegidas: acceder a hardware, crear sockets en puertos bajos, matar procesos de otros usuarios, cargar módulos del kernel, etc. En Linux, el conjunto de privilegios disponibles evolucionó desde el modelo binario UNIX (*root / no-root*) hacia un esquema granular basado en *capabilities*.

1.1.1 Modo usuario vs. modo kernel

El procesador opera en al menos dos niveles de privilegio de hardware:

Definición: Anillos de protección

Los procesadores x86/x86-64 implementan cuatro *rings* (0–3). Linux utiliza sólo dos: **Ring 0** (modo kernel) y **Ring 3** (modo usuario). ARM los denomina *EL1* y *EL0* respectivamente.

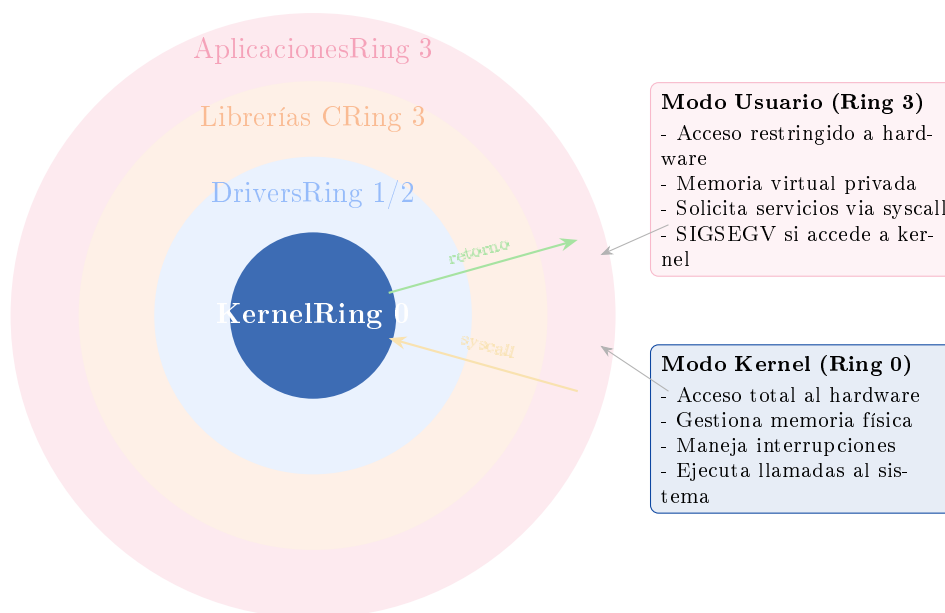


Figure 1.1: Anillos de protección: el kernel en Ring 0, las aplicaciones en Ring 3. La transición se produce mediante *syscalls*.

La interfaz syscall

Cada vez que una aplicación necesita un recurso privilegiado (abrir un archivo, crear un socket, enviar una señal), ejecuta una **llamada al sistema**. La CPU conmuta al Ring 0, el kernel valida el pedido contra las credenciales del proceso y devuelve el resultado.

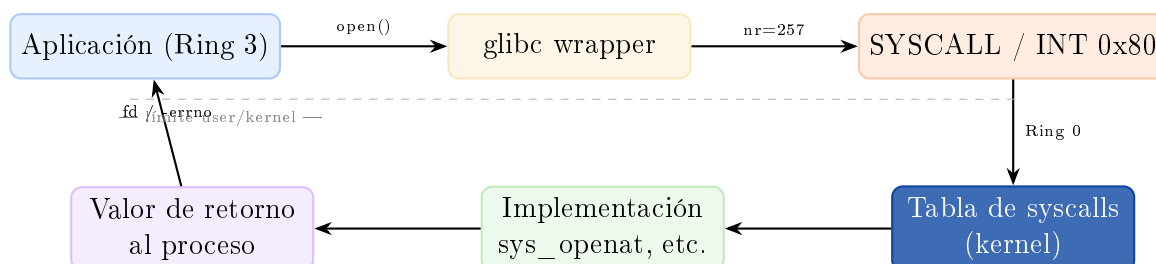


Figure 1.2: Flujo de una syscall: de la aplicación al kernel y retorno

```
Ver syscalls de un proceso en tiempo real
strace -e openat,read,write ls /etc 2>&1 | head -12

\# Contar syscalls de un binario
strace -c cat /etc/hostname

\# Ver el numero de syscall para 'read' en x86-64
grep -w read /usr/include/asm/unistd_64.h
\# define __NR_read 0
```

Implicaciones de seguridad

- Un proceso en modo usuario **no puede** acceder directamente a memoria de otro proceso ni al espacio del kernel.
- El kernel verifica las **credenciales** del proceso en cada syscall sensible antes de ejecutarla.
- Vulnerabilidades de *privilege escalation* suelen explotar errores en el camino de validación dentro del kernel (p.ej. race conditions en syscalls).
- **seccomp** (Secure Computing Mode) permite filtrar qué syscalls puede invocar un proceso, reduciendo la superficie de ataque.

```
Ver el filtro seccomp de un proceso (Docker, por ejemplo)
cat /proc/$(pgrep nginx | head -1)/status | grep Seccomp
  \# Seccomp: 2      (0=desactivado, 1=strict, 2=bpf filtro)

\# Modo strict: solo read/write/exit/_exit/sigreturn
prctl(PR_SET_SECCOMP, SECCOMP_MODE_STRICT);
```

1.1.2 Identidades y credenciales en el kernel: UID/GID y struct cred

Cada proceso en Linux lleva asociado un objeto de credenciales (**struct cred**) mantenido por el kernel. Este objeto concentra toda la información de identidad y privilegio del proceso.

Objeto de credenciales del proceso (por proceso, copy-on-write)

```
struct cred (include/linux/cred.h)

uid_t uid;      /* UID real */
uid_t euid;     /* UID efectivo (determina acceso) */
uid_t suid;     /* UID guardado (para restaurar) */
uid_t fsuid;    /* UID para operaciones de FS */

gid_t gid, egid, sgid, fsgid;

kernel_cap_t cap_inheritable; /* caps heredables */
kernel_cap_t cap_permitted; /* caps permitidas */
kernel_cap_t cap_effective; /* caps activas */
kernel_cap_t cap_bset; /* bounding set */
kernel_cap_t cap_ambient; /* ambient set (Linux 4.3+) */

struct user_namespace *user_ns;
struct group_info *group_info; /* GIDs suplementarios */
```

Figure 1.3: Campos principales de **struct cred** en el kernel Linux

Los cuatro UIDs

Table 1.1: Los cuatro UIDs del proceso y su rol

Campo	Nombre	Propósito
uid	UID real	Quién lanzó el proceso; permanece fijo
euid	UID efectivo	El que el kernel compara contra propietarios de archivos y otros objetos
suid	UID guardado	Copia del euid anterior al cambio; permite restaurar privilegios temporalmente
fsuid	UID de FS	Usado exclusivamente en comprobaciones del sistema de archivos (heredado de SVR4)

euid = 0 ⇒ root

Históricamente, cualquier proceso con `euid == 0` tiene privilegio total. Las *capabilities* POSIX (ver § 1.3) refinan este modelo sin romper la compatibilidad.

Transición de UID: SUID y `setuid()`

Cuando se ejecuta un binario con el bit SUID activo, el kernel copia el `uid` del propietario del archivo en el `euid` y `suid` del nuevo proceso:

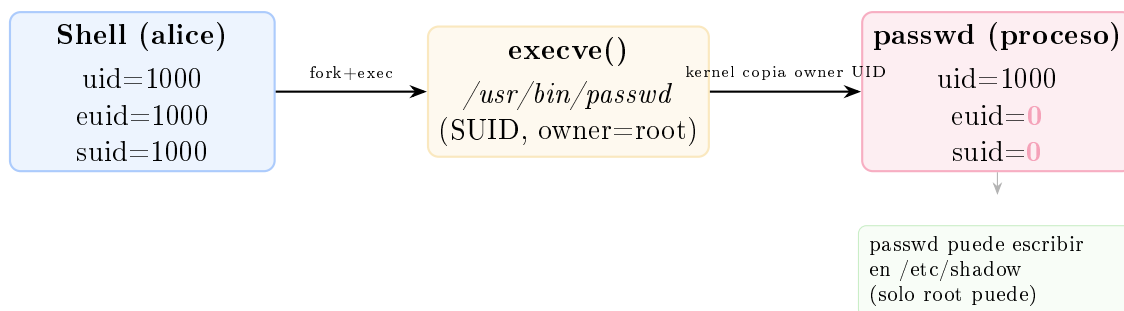


Figure 1.4: Transición de UID al ejecutar un binario SUID: alice ejecuta `passwd` que obtiene `euid=0`

Terminal

```

Verificar credenciales del proceso actual
id
uid=1000(alice) gid=1000(alice) groups=1000(alice),27(sudo)

cat /proc/$$/status | grep -E "^(Uid|Gid):"
Uid:  1000   1000   1000   1000
Gid:  1000   1000   1000   1000
  \# Columnas: real    efectivo guardado  FS

  \# Ver credenciales de otro proceso (requiere permiso o root)
  
```

```
cat /proc/$(pgrep sshd | head -1)/status | grep -E "(Uid|Gid|Cap)"
```

```
/* Cambio de UID desde código C */
#include <sys/types.h>
#include <unistd.h>

/* Bajar privilegios definitivamente (sin vuelta atrás) */
if (setuid(1000) < 0) {
    perror("setuid");
    exit(1);
}

/* Cambio temporal: guarda euid en suid */
seteuid(1000); /* ahora euid=1000, suid=0 (si eramos root) */
/* ... operacion no privilegiada ... */
seteuid(0); /* restauramos privilegio gracias al suid=0 */
```

GIDs suplementarios

Además del GID primario, un proceso puede pertenecer a múltiples grupos suplementarios almacenados en `group_info`. El kernel comprueba todos ellos al evaluar permisos de archivo.

```
Ver todos los grupos del proceso actual
cat /proc/$$/status | grep Groups
Groups: 27 1000 1001 1002

# Equivalente con herramienta de usuario
id
uid=1000(alice) gid=1000(alice) groups=1000(alice),27(sudo),1001(devteam)

# Ejecutar shell en un grupo específico (sin salir del actual)
newgrp docker
```

1.1.3 Capacidades POSIX (*POSIX Capabilities*)

Antes de las capacidades, el modelo era binario: o eres `root` (acceso total) o no puedes hacer nada privilegiado. Linux 2.2 introdujo las **POSIX Capabilities**: una descomposición de los privilegios de `root` en unidades independientes que se pueden otorgar individualmente.

Definición: Capability

Una *capability* es un token que autoriza a un proceso a realizar una operación privilegiada específica, sin necesitar `euid=0`. Hay actualmente **41 capabilities** definidas en el kernel (`include/uapi/linux/capability.h`).

Conjuntos de capabilities

Cada proceso mantiene **cinco conjuntos** de capabilities:

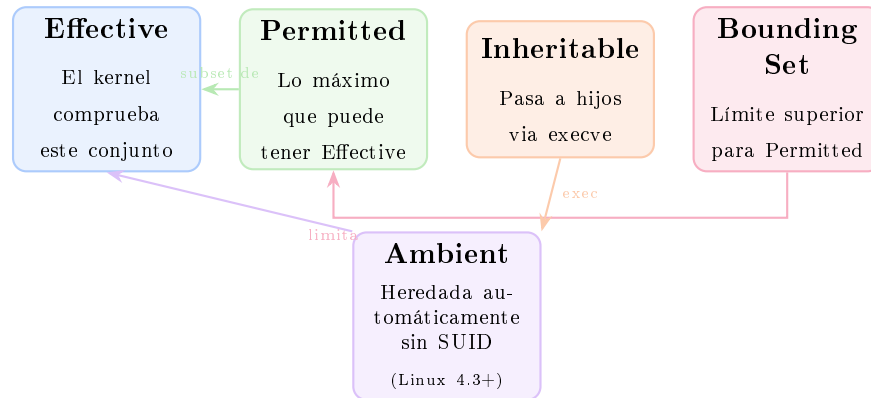


Figure 1.5: Los cinco conjuntos de capabilities de un proceso y sus relaciones

Capabilities más relevantes

Table 1.2: Capabilities POSIX más importantes en Linux

Capability	Permite
CAP_CHOWN	Cambiar propietario de cualquier archivo
CAP_DAC_OVERRIDE	Ignorar comprobaciones de permisos de archivos (DAC)
CAP_DAC_READ_SEARCH	Ignorar bits r/x en directorios para búsquedas
CAP_FOWNER	Operar como propietario de cualquier archivo
CAP_KILL	Enviar señales a procesos de otros usuarios
CAP_NET_BIND_SERVICE	Hacer bind en puertos TCP/UDP < 1024
CAP_NET_RAW	Crear sockets RAW e ICMP
CAP_SYS_ADMIN	Amplio: mount, namespaces, ptrace, setdomain-name...
CAP_SYS_CHROOT	Ejecutar chroot()
CAP_SYS_PTRACE	Trazar cualquier proceso (ptrace)
CAP_SYS_MODULE	Cargar y descargar módulos del kernel
CAP_SYS_TIME	Modificar el reloj del sistema
CAP_SETUID	Hacer setuid() arbitrario
CAP_SETPCAP	Transferir capabilities al bounding set de otros
CAP_AUDIT_WRITE	Escribir en el log de auditoría del kernel

CAP_SYS_ADMIN: la nueva root

CAP_SYS_ADMIN agrupa más de 200 operaciones distintas y se considera el equivalente práctico de root. Otorgarla a un proceso o contenedor debe tratarse con el mismo cuidado que otorgar `uid=0`.

Capabilities en archivos

Los capabilities también pueden asociarse a **archivos ejecutables** (en `xattr security.capability`), evitando el bit SUID:

```
Ver capabilities de un binario
getcap /usr/bin/ping
/usr/bin/ping cap_net_raw=ep

\# Asignar capability a un binario propio (requiere CAP_SETFCAP)
setcap cap_net_bind_service=+ep /usr/local/bin/mi_servidor

\# Quitar todas las capabilities de un binario
setcap -r /usr/local/bin/mi_servidor

\# Ver capabilities del proceso actual
capsh --print

\# Ver capabilities de un proceso por PID
cat /proc/$(pgrep ping | head -1)/status | grep Cap
CapInh: 0000000000000000
CapPrm: 00000000000002000 \# CAP_NET_RAW = bit 13
CapEff: 00000000000002000
CapBnd: 000001fffffffffff
CapAmb: 0000000000000000

\# Decodificar el valor hexadecimal
capsh --decode=00000000000002000
0x00000000000002000=cap_net_raw
```

Regla de transformación en `execve()`

Al ejecutar un nuevo binario, el kernel recalcula los conjuntos del proceso hijo usando la siguiente fórmula:

```
P'(permitted) = (P(inheritable) & F(inheritable)) | (F(permitted) & P(bounding)) | P(ambient)
P'(effective) = F(effective) ? P'(permitted) : P(ambient)
P'(inheritable) = P(inheritable)

P = proceso padre, F = atributos del archivo, P' = proceso hijo tras execve()
```

Figure 1.6: Fórmulas de transformación de capabilities al ejecutar un programa

Principio de mínimo privilegio con capabilities

En lugar de ejecutar un servicio como root, asigna solo las capabilities que necesita. Por ejemplo, un servidor web que escucha en el puerto 80:

```
setcap cap_net_bind_service=+ep /usr/sbin/nginx
```

Así, si el proceso es comprometido, el atacante no obtiene CAP_SYS_ADMIN ni otras capabilities peligrosas.

1.1.4 Permisos, ACL y atributos extendidos

El control de acceso discrecional (DAC) de Linux opera en tres capas superpuestas: los **permisos UNIX clásicos**, las **ACL POSIX** (granularidad por usuario/grupo adicional) y los **atributos extendidos** (xattrs), que soportan tanto ACL como metadatos de seguridad (SELinux, capabilities).

Permisos UNIX clásicos

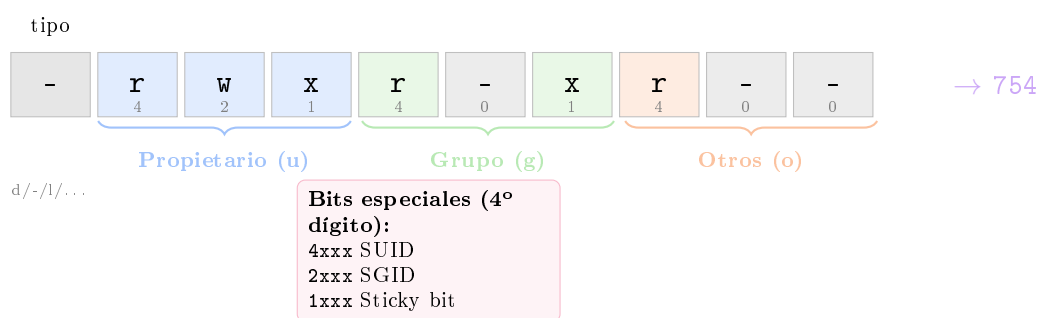


Figure 1.7: Estructura de permisos `-rwxr-xr-` (octal 754) con los tres conjuntos

Table 1.3: Algoritmo de evaluación de permisos DAC

Paso	Condición	Acción
1	<code>euid == 0</code> (root)	Acceso concedido (salvo <code>x</code> en archivos)
2	<code>euid == uid(archivo)</code>	Aplicar bits del propietario
3	<code>egid</code> o GID suplementario == <code>gid(archivo)</code>	Aplicar bits del grupo
4	Ninguno de los anteriores	Aplicar bits de “otros”

Terminal

```
Examinar permisos completos de un archivo
stat /etc/sudoers
  Acceso: (0440/-r--r----)  Uid: (0/root)  Gid: (0/root)

\# Cambiar permisos
chmod 640 /etc/mi_config.conf      \# propietario rw, grupo r
chmod u+s /usr/local/bin/binario  \# activar SUID
chmod 1777 /tmp/compartido         \# sticky bit
```

```
\# umask: permisos restados al crear archivos
umask 027 \# archivos nuevos = 640, directorios = 750
```

ACL POSIX

Las ACL POSIX permiten asignar permisos a usuarios y grupos **adicionales** al propietario y al grupo del archivo, superando la limitación del modelo clásico.

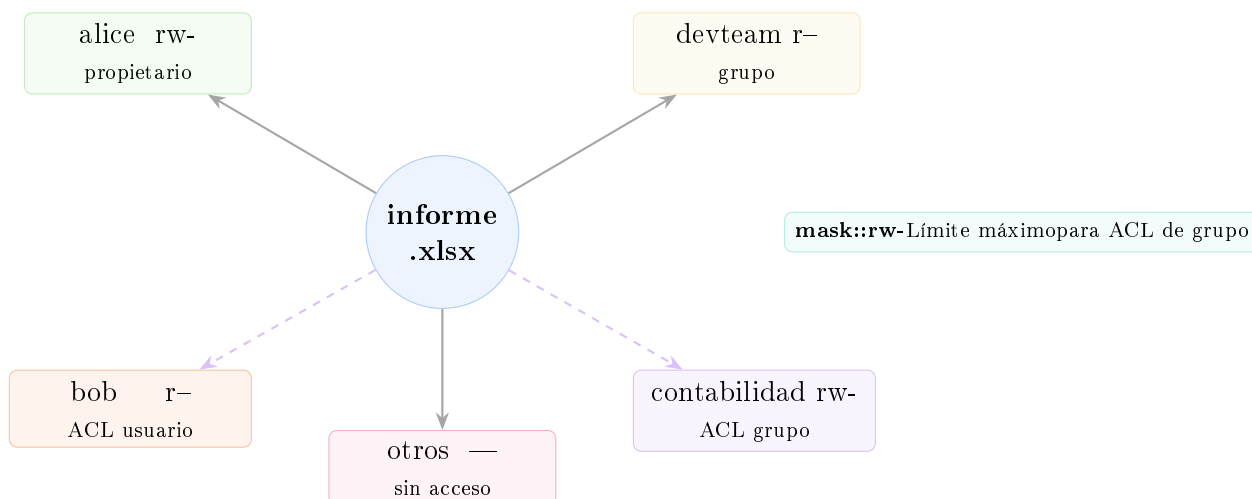


Figure 1.8: Modelo de ACL POSIX: entradas estándar (flechas sólidas) y entradas extendidas (flechas punteadas)

```

Ver ACL de un archivo
getfacl informe.xlsx
\# file: informe.xlsx
\# owner: alice
\# group: devteam
user::rw-
user:bob:r--
group::r--
group:contabilidad:rw-
mask::rw-
other::---

\# Agregar entrada ACL de usuario
setfacl -m u:bob:r-- informe.xlsx

\# Agregar ACL de grupo
setfacl -m g:contabilidad:rw- informe.xlsx

\# ACL por defecto en directorio (se heredan al crear archivos)
setfacl -d -m g:devteam:rwx /shared/proyectos/

\# Eliminar todas las ACL extendidas
setfacl -b informe.xlsx

\# Copiar ACL de un archivo a otro
getfacl src.txt | setfacl --set-file=- dst.txt

```

La máscara de ACL

La entrada `mask` limita los permisos efectivos de todas las entradas ACL de grupo y de usuarios adicionales (pero **no** al propietario ni a “otros”). Se recalcula automáticamente con `chmod` si no se usa `-n`.

Atributos extendidos (xattrs)

Los **xattrs** son pares clave-valor asociados a inodes del sistema de archivos. Son la infraestructura que soporta ACL POSIX, capabilities de archivo, etiquetas SELinux/AppArmor y metadatos personalizados.

Table 1.4: Espacios de nombres de atributos extendidos

Namespace	Acceso	Uso
<code>user.*</code>	Cualquier usuario	Metadatos de aplicación
<code>system.*</code>	Kernel / root	ACL POSIX (<code>system.posix_acl_*</code>)
<code>security.*</code>	Root / LSM	SELinux labels, capabilities (<code>security.capability</code>)
<code>trusted.*</code>	Solo root	Metadatos del sistema de confianza

```

Ver todos los xattrs de un archivo
getfattr -d -m "" /usr/bin/ping
  \# file: usr/bin/ping
  security.capability=0s...  (capabilities codificadas en base64
  )

\# Ver xattr de seguridad SELinux
getfattr -n security.selinux /etc/passwd
  security.selinux="system_u:object_r:passwd_file_t:s0"

\# Establecer xattr de usuario
setfattr -n user.descripcion -v "archivo confidencial" reporte.
pdf
getfattr -n user.descripcion reporte.pdf
  user.descripcion="archivo confidencial"

\# Atributos inmutables (chattr / lsattr)
chattr +i /etc/resolv.conf      \# inmutable: ni root puede
  modificarlo
lsattr /etc/resolv.conf
  ----i-----e-- /etc/resolv.conf
chattr -i /etc/resolv.conf      \# quitar inmutabilidad

```

1.1.5 Protecciones relacionadas: nosuid, noexec, nodev

Las opciones de montaje son una capa de defensa adicional que restringe los privilegios que puede ejercer el contenido de una partición o sistema de archivos, independientemente de los permisos de los archivos individuales.



Figure 1.9: Las tres opciones de montaje de seguridad y sus efectos

```
Ver opciones de montaje actuales
cat /proc/mounts | grep -E "nosuid|noexec|nodev"
tmpfs /tmp tmpfs rw,nosuid,nodev,noexec 0 0
/dev/sda2 /home ext4 rw,nosuid,nodev 0 0

\# Montar con opciones de seguridad (como root)
mount -o nosuid,noexec,nodev /dev/sdb1 /mnt/usb

\# Agregar a /etc/fstab para que sea permanente
\# /dev/sda3 /var ext4 defaults,nosuid,noexec,nodev 0 2

\# Remontar con nuevas opciones sin desmontar
mount -o remount,nosuid,noexec /tmp

\# Verificar que noexec funciona
cp /usr/bin/ls /tmp/test_ls
/tmp/test_ls
bash: /tmp/test_ls: Permission denied (noexec activo)
```

Configuración recomendada en /etc/fstab

```
# Punto de montaje seguro para /tmp
tmpfs /tmp tmpfs nosuid,noexec,nodev,size=2G 0 0

# /home: no necesita dispositivos ni SUID
/dev/sda3 /home ext4 defaults,nosuid,nodev 0 2

# /var: logs y uploads, sin ejecución directa
/dev/sda4 /var ext4 defaults,nosuid,noexec,nodev 0 2

# /boot: solo lectura, todos los flags
/dev/sda1 /boot ext4 ro,nosuid,noexec,nodev 0 2

# /usr: ejecutables del sistema, SUID si necesario
/dev/sda2 /usr ext4 defaults,nodev 0 2
```

Figure 1.10: Ejemplo de /etc/fstab con opciones de seguridad

Protección adicional: bind mounts y contenedores

```
Bind mount de solo lectura (util en contenedores)
mount --bind /etc /mnt/etc_ro
mount -o remount,ro,nosuid,noexec,nodev /mnt/etc_ro

\# Docker aplica estas opciones por defecto en tmpfs internos
docker run --tmpfs /tmp:nosuid,noexec,nodev ubuntu bash

\# Ver opciones efectivas de namespaces de montaje
cat /proc/self/mountinfo | awk '{print $5, $6}' | column -t |
head -20
```

Eludir noexec con intérpretes

noexec impide ejecutar binarios directamente, pero **no** impide hacer: `python3 /tmp/malware.py` o `bash /tmp/script.sh`. La protección completa requiere combinar noexec con controles sobre intérpretes (AppArmor, SELinux, seccomp) y monitorización.

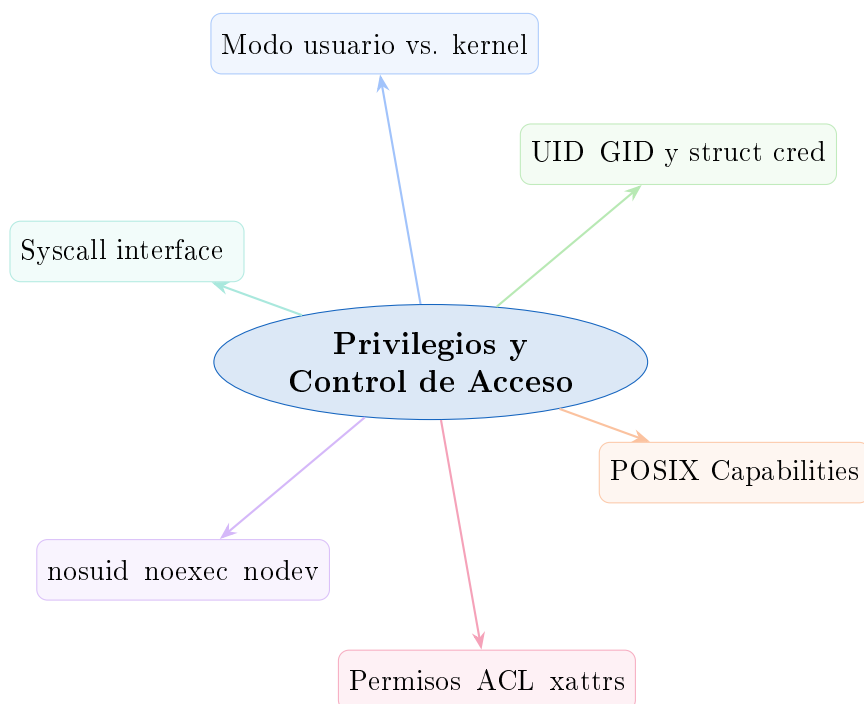
Resumen del capítulo

Figure 1.11: Mapa conceptual del capítulo

Principios clave a recordar

1. **Modo kernel vs. usuario:** la CPU impone la separación; el kernel valida credenciales en cada syscall.
2. **euid determina el acceso:** de los cuatro UIDs, es el **euid** el que el kernel comprueba contra los objetos.
3. **Capabilities granularizan root:** prefiere asignar sólo las capabilities necesarias en lugar de otorgar **euid=0**.
4. **ACL extienden el DAC:** cuando necesites permisos para más de un usuario/grupo adicional, usa ACL en lugar de ampliar el bit “otros”.
5. **nosuid + noexec + nodev** en **/tmp** y **/home** es la configuración mínima recomendada en cualquier sistema de producción.